
Projet d'IHM

Sudoku

VB
.net

Table des matières

I.	Introduction :	3
II.	Schéma d'ordonnancement :	5
III.	Les formulaire s et leur code :	6
1.	Formulaire d'accueil.....	6
2.	Formulaire statistique :	9
3.	Formulaire grille de jeu :	13
IV.	Les modules et leur code :	17
1.	Module gestionnaire des données :	17
2.	Module gestionnaire de partie :	19
3.	Module Génération automatique de la grille :	20
V.	Représentation de l'enregistrement des informations dans le fichier :.....	21

I. Introduction :

L'application à développer est un Sudoku en VB.net. Cette application permet à un joueur de jouer au sudoku tout en sauvegardant ses données à savoir son nom, le nombre de parties jouées, son meilleur temps et le temps total de jeu.

Lorsque le joueur désire jouer, une grille de 9*9 cases est générée avec des chiffres placés aléatoirement afin de commencer la partie. Lorsque la grille est générée, un timer est lancé, avec un temps limite. À la fin de ce temps la partie se termine, et une vérification automatique de la grille est effectuée et indique au joueur si la partie est gagnée ou perdue. Aussi, si toutes les cases sont remplies, la grille est vérifiée automatiquement et la fin de partie est activée. Si deux ou plus mêmes valeurs sont présentes sur la même ligne, colonne, ou secteur, les cases concernées par l'erreur sont automatiquement colorées en rouge.

Un bouton retour / abandonner est présent sur le **formulaire** du jeu. Si le joueur abandonne la partie, une première **messageBox** de confirmation demande au joueur s'il est sûr de vouloir quitter la partie. Si le joueur clique sur « non » alors la partie continue et le timer (qui était en pause) redémarre ; sinon une **messageBox** apparaît en précisant que la partie est perdue, puis au clic du bouton « ok » le **formulaire** d'accueil s'ouvre et celui de la grille disparaît. Le nom du **formulaire** contient le nom du joueur qui joue de la partie.

Le **formulaire** d'accueil permet d'entrer le nom du joueur avec lequel nous voulons jouer, créer un nouveau joueur ou sélectionner le joueur désiré. Si le joueur qu'on essaie d'ajouter est déjà présent dans la liste, une **messageBox** d'erreur s'affiche précisant que le joueur existe déjà. Si le nom de joueur contient des espaces alors une **messageBox** s'affiche indiquant que le nom entré est incorrect. Lorsqu'un joueur est créé, ses statistiques sont 0 pour le nombre de parties, 0 pour le temps total passé et 999 pour son meilleur temps de partie afin de permettre un tri des joueurs dans l'écran statistique consistant. Pour finir une croix permet de supprimer n'importe quel joueur de la liste, une **messageBox** de confirmation de suppression avec le nom du joueur s'affiche (sauf le joueur invité, créé de base. Si on essaie de le supprimer une **messageBox** s'affiche précisant que le l'invité ne peut être supprimé). Un bouton « quitter » provoque l'ouverture d'une **messageBox** qui demande confirmation, si « oui » est sélectionné alors l'arrêt complet de l'application est provoqué ainsi que la sauvegarde des données. De plus, tous les formulaires cités, lors de leur

fermeture, qu'elle soit à partir d'un bouton ou de la croix de la zone de contrôle, les données sont automatiquement sauvegardées dans le fichier.

Depuis l'accueil est aussi possible d'ouvrir un autre **formulaire** permettant de consulter les statistiques de chaque joueur en cliquant sur un bouton qui affiche un nouveau **formulaire** affichant dans deux **listBox** différentes les joueurs et leur meilleur temps. Lorsqu'on sélectionne un joueur ou un meilleur score de leur liste (en cliquant directement sur la **listBox** ou en utilisant la **comboBox** présente), le focus des **listBox** s'alignent ainsi que le joueur choisi dans la **listBox**. Deux boutons offrent la possibilité de trier les listes par ordre croissant ou décroissant des joueurs. Un autre bouton permet de les trier selon l'ordre croissant ou décroissant des scores. Pour passer d'un tri croissant à décroissant, il suffit de presser à nouveau le bouton de tri. Un bouton permet de revenir au tri original (donc l'ordre dans lequel les joueurs ont été ajoutés). Pour finir un bouton permet d'afficher les statistiques complètes du joueur sélectionné dans une **messageBox** affichant le nombre de parties, le temps de la meilleure partie ainsi que le temps total de jeu de ce joueur. Il est enfin possible de revenir à l'accueil à l'aide d'un bouton « retour à l'accueil ».

Lorsqu'on lance une partie le choix de trois difficulté est possible entre :

- Facile (le temps de la partie est de 500 secondes et 20 chiffres générés automatiquement sont placés sur la grille)
- Moyen (le temps de la partie est de 120 secondes et 35 chiffres générés automatiquement sont placés sur la grille)
- Difficile (le temps de la partie est de 60 secondes et 50 chiffres générés automatiquement sont placés sur la grille)

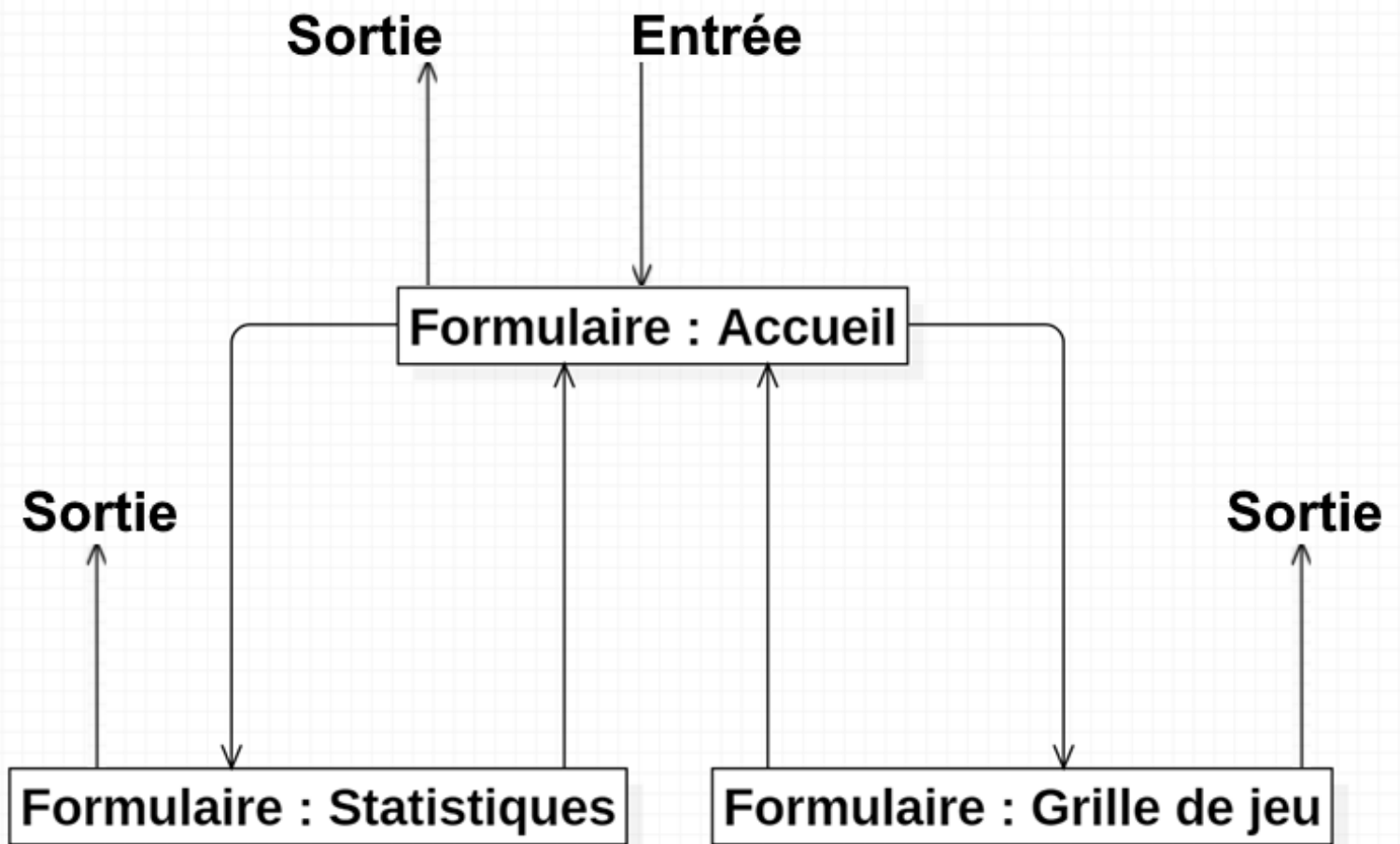
La saisie de chaîne de caractère autres que des chiffres et du chiffre 0 ne font aucun effet sur la grille, seul les chiffres de 1 à 9 sont autorisés.

Pour finir, à chaque arrêt de partie (que ce soit par la croix, ou par le bouton quitter) les statistiques du joueur sont modifiées (le nombre de partie jouée est incrémenté et le temps passé sur le formulaire s'additionne avec le temps passé total sur le jeu.)

Une croix en haut à droite est présente tous les **formulaires** et permet l'arrêt complet de l'application tout en sauvegardant automatiquement les données dans le fichier. Ce fichier, si l'utilisateur altère manuellement des lignes de données, celles-ci ne seront pas lues au chargement suivant du programme mais cela n'empêchera pas au programme de lire les lignes correctes du fichier.

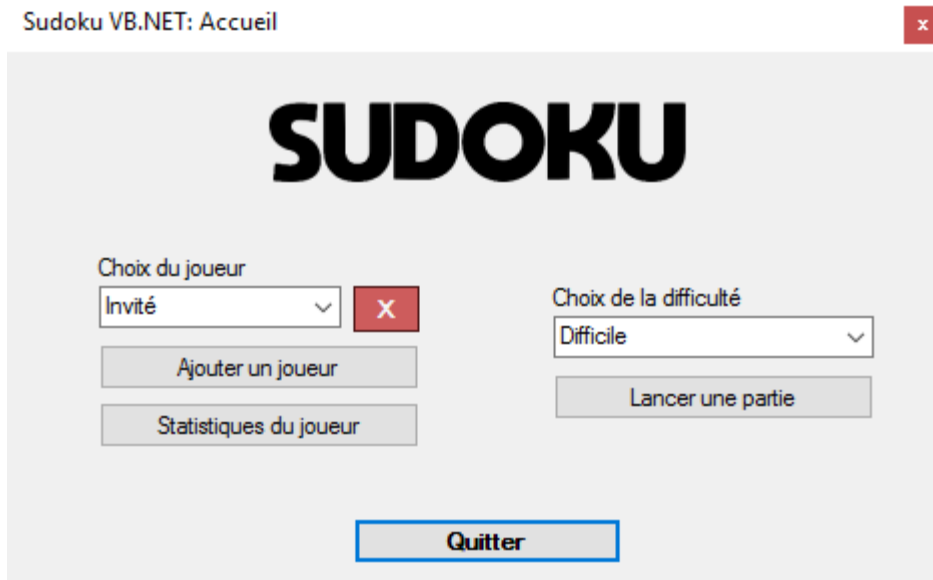


II. Schéma d'ordonnement :



III. Les formulaires et leur code :

I. Formulaire d'accueil



```
Imports System.ComponentModel
Imports System.Drawing.Text
Public Class Accueil
    Public CustomFont As New PrivateFontCollection()
    Private Sub Accueil_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        GestionnaireDeDonnées.ChargementDonnées()
        Me.Update_CBBBox_NomsJoueurs()
        Me.ComboBox_ChoixDifficulté.Items.AddRange({Hard, Average, Easy})
        Me.Ranger()
        Me.ComboBox_ChoixJoueur.DropDownHeight = Me.ComboBox_ChoixJoueur.ItemHeight *
10
        CustomFont.AddFontFile("./Ressources/moon_get-Heavy.ttf")
    End Sub

    Private Sub Button_Quitter_Click(sender As Object, e As EventArgs) Handles
Button_Quitter.Click
        Dim res As Integer = MsgBox("Êtes-vous sûr de vouloir quitter ?",
MsgBoxStyle.YesNo, "Quitter")
        If (res = MsgBoxResult.Yes) Then
            GestionnaireDeDonnées.SauvegardeDonnées()
            End
        End If
    End Sub

    Private Sub Button_LancerPartie_Click(sender As Object, e As EventArgs) Handles
Button_LancerPartie.Click
        If Not Me.ComboBox_ChoixJoueur.Items.Contains(Me.ComboBox_ChoixJoueur.Text)
Then
            MsgBox("Vous devez choisir un joueur pour lancer une partie",
MsgBoxStyle.OkOnly, "Avertissement")
            Exit Sub
        End If
    End Sub
End Class
```



```
If Not
Me.ComboBox_ChoixDifficulté.Items.Contains(Me.ComboBox_ChoixDifficulté.Text) Then
    MsgBox("Vous devez choisir une difficulté pour lancer une partie",
MsgBoxStyle.OkOnly, "Avertissement")
    Exit Sub
End If
GestionnaireDePartie.LancerPartie(Me.ComboBox_ChoixJoueur.Text,
Me.ComboBox_ChoixDifficulté.Text)
Me.Hide()
End Sub

Private Sub Button_AjoutJoueur_Click(sender As Object, e As EventArgs) Handles
Button_AjoutJoueur.Click
    Dim nomJoueur As String = Me.ComboBox_ChoixJoueur.Text
    If Me.ComboBox_ChoixJoueur.Items.Contains(Me.ComboBox_ChoixJoueur.Text) Then
        MsgBox("Joueur déjà existant", MsgBoxStyle.OkOnly, "avertissement")
        Exit Sub
    ElseIf Me.ComboBox_ChoixJoueur.Text.Contains(" ") Then
        MsgBox("Nom de joueur incorrect", MsgBoxStyle.OkOnly, "avertissement")
        Exit Sub
    ElseIf GestionnaireDeDonnées.MaxJoueurAtteint Then
        MsgBox("Nombre de joueurs maximal atteint (100)", MsgBoxStyle.OkOnly,
"Avertissement")
        Exit Sub
    End If
    GestionnaireDeDonnées.AjouterJoueur(nomJoueur)
    Me.Update_CBBox_NomsJoueurs()
End Sub

Private Sub Update_CBBox_NomsJoueurs()
    Dim PlayerData = GestionnaireDeDonnées.GetDonnées
    Me.ComboBox_ChoixJoueur.Items.Clear()
    For Each element As Joueur In PlayerData
        Me.ComboBox_ChoixJoueur.Items.Add(element.Nom)
    Next
End Sub

Private Sub Accueil_FormClosing(sender As Object, e As FormClosingEventArgs)
Handles Me.FormClosing
    GestionnaireDeDonnées.SauvegardeDonnées()
End Sub

Private Sub Button_StatistiquesJoueur_Click(sender As Object, e As EventArgs)
Handles Button_StatistiquesJoueur.Click
    Me.Hide()
    Statistiques.Show()
End Sub

Private Sub Ranger() Handles Me.VisibleChanged
    Me.CenterToScreen()
    Me.ComboBox_ChoixJoueur.SelectedIndex =
Me.ComboBox_ChoixDifficulté.SelectedIndex = 0
    Me.ComboBox_ChoixDifficulté.Text = Me.ComboBox_ChoixDifficulté.Items(0)
    Me.ComboBox_ChoixJoueur.Text = Me.ComboBox_ChoixJoueur.Items(0)
End Sub

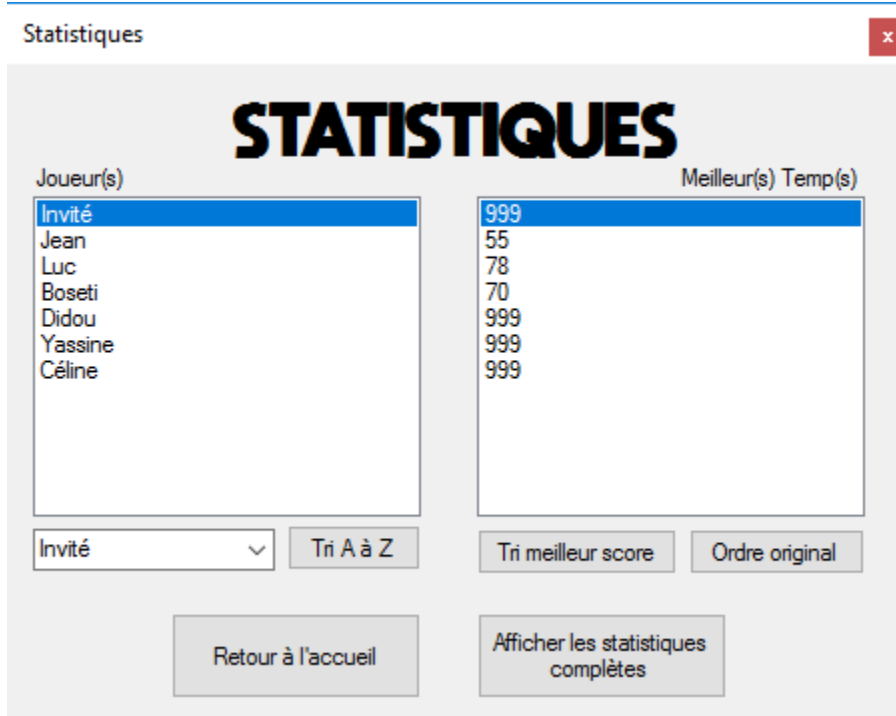
Private Sub ComboBox_ChoixJoueur_SelectedIndexChanged(sender As Object, e As
System.Windows.Forms.KeyPressEventArgs) Handles ComboBox_ChoixJoueur.KeyPress
    If e.KeyChar = ChrW(Keys.Enter) Then
        Me.Button_AjoutJoueur.PerformClick()
```



```
End If
End Sub

Private Sub Button_SuppressionJoueur_Click(sender As Object, e As EventArgs)
Handles Button_SuppressionJoueur.Click
    Dim Nom_Supr = Me.ComboBox_ChoixJoueur.Text
    If Nom_Supr = GetDonnées(0).Nom Then
        MsgBox("Vous ne pouvez pas supprimer l'invité", MsgBoxStyle.OkOnly,
"Avertissement")
        Exit Sub
    End If
    Dim res As Integer = MsgBox("Êtes-vous sûr de vouloir supprimer le joueur " &
Nom_Supr & " ?", MsgBoxStyle.YesNo, "Avertissement")
    If (res = MsgBoxResult.Yes) Then
        GestionnaireDeDonnées.SupprimerJoueur(Nom_Supr)
        Me.Update_CBBox_NomsJoueurs()
        Me.ComboBox_ChoixJoueur.SelectedIndex = 0
    End If
End Sub
End Class
```


2. Formulaire statistique :



```
Public Class Statistiques
```

```
    Private TriAZ = False
    Private Tri09 = False
```

```
    Private Sub Statistiques_Load(sender As Object, e As EventArgs) Handles
MyBase.Load
        Me.CenterToScreen()
        Me.ResetDonnées()
        Me.Label_TitreStat.Font = New Font(Accueil.CustomFont.Families(0), 20,
FontStyle.Regular)
        AddHandler ListBox_Joueurs.SelectedIndexChanged, AddressOf SynchroSelect
        AddHandler ListBox_MeilleursTemps.SelectedIndexChanged, AddressOf
SynchroSelect
        AddHandler ComboBox_ChoixJoueur.SelectedIndexChanged, AddressOf SynchroSelect
        Me.ComboBox_ChoixJoueur.DropDownHeight = Me.ComboBox_ChoixJoueur.ItemHeight *
10
    End Sub

    Private Sub SynchroSelect(sender As Object, e As EventArgs)
        For Each element In Me.Controls
            If {GetType(ListBox), GetType(ComboBox)}.Contains(element.GetType) Then
                element.SelectedIndex() = sender.SelectedIndex()
            End If
        Next
        Me.ComboBox_ChoixJoueur.Text = ListBox_Joueurs.SelectedItem
    End Sub

    Public Sub ResetDonnées()
```



```
TriAZ = False
Tri09 = False
ListBox_Joueurs.Items.Clear()
ListBox_MeilleursTemps.Items.Clear()
ComboBox_ChoixJoueur.Items.Clear()
For Each element In GestionnaireDeDonnées.GetDonnées
    ListBox_Joueurs.Items.Add(element.Nom)
    ListBox_MeilleursTemps.Items.Add(element.MeilleurTemps)
    ComboBox_ChoixJoueur.Items.Add(element.Nom)
Next
Me.ListBox_Joueurs.SelectedIndex = Me.ListBox_MeilleursTemps.SelectedIndex =
Me.ComboBox_ChoixJoueur.SelectedIndex = 0

End Sub

Private Sub Statistiques_FormClosing(sender As Object, e As FormClosingEventArgs)
Handles Me.FormClosing
    GestionnaireDeDonnées.SauvegardeDonnées()
End
End Sub

Private Sub Button_FullStats_Click(sender As Object, e As EventArgs) Handles
Button_FullStats.Click
    Dim Stats As Joueur = GetJoueur(Me.ListBox_Joueurs.SelectedItem)
    Dim Temps_Total = Stats.TempsTotal \ 60 & "mn" & Stats.TempsTotal Mod 60 & "s"
    MsgBox("Nombre de parties: " & Stats.NbParties & vbCrLf & "Meilleur partie: "
&
        Stats.MeilleurTemps & "s" & vbCrLf & "Temps total: " & Temps_Total,
MsgBoxStyle.OkOnly,
    "Statistiques Complètes de " & Stats.Nom)
End Sub

Private Sub Button_RetourAccueil_Click(sender As Object, e As EventArgs) Handles
Button_RetourAccueil.Click
    Me.Hide()
    Accueil.Show()
End Sub

Private Sub Centrer() Handles Me.VisibleChanged
    Me.CenterToScreen()
End Sub

Private Sub MajALAffichage() Handles Me.VisibleChanged
    Me.ResetDonnées()
End Sub

Private Sub Button_TriAlpha_Click(sender As Object, e As EventArgs) Handles
Button_TriAlpha.Click
    Me.TriAZ = Not Me.TriAZ
    If Me.TriAZ Then
        For i As Integer = Me.ListBox_Joueurs.Items.Count - 1 To 1 Step -1
            For j As Integer = 0 To i - 1
                If ListBox_Joueurs.Items(j) > ListBox_Joueurs.Items(j + 1) Then
                    Dim tempJ = ListBox_Joueurs.Items(j), tempMT =
ListBox_MeilleursTemps.Items(j)
                    Dim CBboxJ = ComboBox_ChoixJoueur.Items(j)
                    ListBox_Joueurs.Items.RemoveAt(j)
                    ListBox_Joueurs.Items.Insert(j + 1, tempJ)
                    ListBox_MeilleursTemps.Items.RemoveAt(j)
                    ListBox_MeilleursTemps.Items.Insert(j + 1, tempMT)
                    ComboBox_ChoixJoueur.Items.RemoveAt(j)
```



```
        ComboBox_ChoixJoueur.Items.Insert(j + 1, CBboxJ)
    End If
Next
Next
Else
    For i As Integer = Me.ListBox_Joueurs.Items.Count - 1 To 1 Step -1
        For j As Integer = 0 To i - 1
            If ListBox_Joueurs.Items(j) < ListBox_Joueurs.Items(j + 1) Then
                Dim tempJ = ListBox_Joueurs.Items(j), tempMT =
                ListBox_MeilleursTemps.Items(j)
                Dim CBboxJ = ComboBox_ChoixJoueur.Items(j)
                ListBox_Joueurs.Items.RemoveAt(j)
                ListBox_Joueurs.Items.Insert(j + 1, tempJ)
                ListBox_MeilleursTemps.Items.RemoveAt(j)
                ListBox_MeilleursTemps.Items.Insert(j + 1, tempMT)
                ComboBox_ChoixJoueur.Items.RemoveAt(j)
                ComboBox_ChoixJoueur.Items.Insert(j + 1, CBboxJ)
            End If
        Next
    Next
End If

Me.ComboBox_ChoixJoueur.SelectedIndex = 0
End Sub

Private Sub Button_TriScore_Click(sender As Object, e As EventArgs) Handles
Button_TriScore.Click
    Me.Tri09 = Not Me.Tri09
    If Me.Tri09 Then
        For i As Integer = Me.ListBox_MeilleursTemps.Items.Count - 1 To 1 Step -1
            For j As Integer = 0 To i - 1
                If ListBox_MeilleursTemps.Items(j) >
                ListBox_MeilleursTemps.Items(j + 1) Then
                    Dim tempJ = ListBox_Joueurs.Items(j), tempMT =
                    ListBox_MeilleursTemps.Items(j)
                    Dim CBboxJ = ComboBox_ChoixJoueur.Items(j)
                    ListBox_Joueurs.Items.RemoveAt(j)
                    ListBox_Joueurs.Items.Insert(j + 1, tempJ)
                    ListBox_MeilleursTemps.Items.RemoveAt(j)
                    ListBox_MeilleursTemps.Items.Insert(j + 1, tempMT)
                    ComboBox_ChoixJoueur.Items.RemoveAt(j)
                    ComboBox_ChoixJoueur.Items.Insert(j + 1, CBboxJ)
                End If
            Next
        Next
    Else
        For i As Integer = Me.ListBox_MeilleursTemps.Items.Count - 1 To 1 Step -1
            For j As Integer = 0 To i - 1
                If ListBox_MeilleursTemps.Items(j) <
                ListBox_MeilleursTemps.Items(j + 1) Then
                    Dim tempJ = ListBox_Joueurs.Items(j), tempMT =
                    ListBox_MeilleursTemps.Items(j)
                    Dim CBboxJ = ComboBox_ChoixJoueur.Items(j)
                    ListBox_Joueurs.Items.RemoveAt(j)
                    ListBox_Joueurs.Items.Insert(j + 1, tempJ)
                    ListBox_MeilleursTemps.Items.RemoveAt(j)
                    ListBox_MeilleursTemps.Items.Insert(j + 1, tempMT)
                    ComboBox_ChoixJoueur.Items.RemoveAt(j)
                    ComboBox_ChoixJoueur.Items.Insert(j + 1, CBboxJ)
                End If
            Next
        Next
    End If
```



```
        Next
    Next
End If

    Me.ComboBox_ChoixJoueur.SelectedIndex = 0
End Sub

Private Sub Button_AucunTri_Click(sender As Object, e As EventArgs) Handles
Button_AucunTri.Click
    Me.ResetDonnées()
End Sub
End Class
```

3. Formulaire grille de jeu :

Sudoku VB.NET: Invité à toi de jouer ! x

				7				
								9
9			1					
7					2			
	2					5		
8		9		3			6	1
1								
3	8			9	5	6		
5								

Commandes

Temps restant: 45

[Retour/Abandonner](#)

```
Public Class GrilleDeJeu
    Const TAILLE_GRILLE = 9
    Const Marge = 20, BoxSpacing = 5, BoxDim = 50, TailleFenetreX = 518,
    TailleFenetreY = 578
    Public Timer_Partie As Timer = New Timer()
    Public Grille

    ' Au chargement, initialise les paramètres de bases du formulaires (compteur,
    textBox et En tête)
    Public Sub GrilleDeJeu_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        AddHandler Timer_Partie.Tick, AddressOf MajTemps
        Me.GénérerGrille()
        Me.MajVisible()
        Me.Size = New Size(TailleFenetreX, TailleFenetreY)
    End Sub

    'Vide toutes les TextBox de la grille et remet leur style par défaut
    Public Sub ViderGrille()
        For Each txtBox As TextBox In Grille
```



```
txtBox.Text = ""
txtBox.BackColor = Color.LightGray
txtBox.ForeColor = Color.Black
txtBox.Enabled = True

Next
End Sub

'Remplis la grille avec des valeurs d'une grille valide selon la difficulté
choisie, représentée par
'nbCells, le nombre de cases préremplies
Public Sub RemplirGrille(ByVal nbCells As Integer)
    Static Generator As System.Random = New System.Random()
    'Utilisation d'un code open source afin de générer des grilles correctes
    aléatoirement
    ' - Code provenant d'un projet de générateur de grille sudoku open source
    ' - Par : The ANZAC, 17 Feb 2009
    ' - lien : https://www.codeproject.com/Articles/23206/Sudoku-Algorithm-Generates-a-Valid-Sudoku-in
    SudoGen.GenerateGrid()
    Dim NbCasesRestantes = nbCells
    Dim i As Integer = 0, j As Integer = 0
    For Each s As SudoGen.Square In SudoGen.Sudoku
        If NbCasesRestantes = 0 Then Exit Sub
        Dim PlacerValeur As Boolean = Generator.Next(1, TAILLE_GRILLE *
TAILLE_GRILLE + 1) <= nbCells
        If PlacerValeur Then
            NbCasesRestantes -= 1
            Grille(i, j).Text = s.Value
            Grille(i, j).Enabled = False
            Grille(i, j).BackColor = Color.Coral
            Grille(i, j).ForeColor = Color.White

            End If
            j += 1
            If j = 9 Then
                j = 0
                i += 1
            End If
        End Sub

        Next

        'instancie chaque textbox et leurs paramètres et les range dans une matrice 9x9
        Private Sub GénérerGrille()
            Dim txtBox = New TextBox
            Dim TAB(TAILLE_GRILLE - 1, TAILLE_GRILLE - 1) As TextBox
            Grille = TAB
            Dim dX = 0, dY = 0
            For i = 0 To TAILLE_GRILLE - 1
                If i > 2 And i < 6 Then
                    dY = BoxSpacing
                ElseIf i > 5 Then
                    dY = BoxSpacing * 2
                End If
                For j = 0 To TAILLE_GRILLE - 1
                    txtBox.Name = "txtBox_" & i & j 'txtBox.Name.ToCharArray()(7) = i
                    txtBox.Name.ToCharArray()(8) = j
                    If j > 2 And j < 6 Then
                        dX = BoxSpacing
                    ElseIf j > 5 Then
                        dX = BoxSpacing * 2
                    End If
                End For
            End For
```



```
End If
txtBox.Location = New Point((BoxDim * j) + dX + Marge, (BoxDim * i) +
dY + Marge)
txtBox.Multiline = True
txtBox.Size = New Size(BoxDim, BoxDim)
txtBox.Font = New Font("Trebuchet MS", txtBox.Height / 1.7)
txtBox.MaxLength = 1
txtBox.TextAlign = HorizontalAlignment.Center
Me.Controls.Add(txtBox)
AddHandler txtBox.KeyPress, AddressOf CaractèreAutorisés
AddHandler txtBox.KeyDown, AddressOf PasDeDelete
Grille(i, j) = txtBox
txtBox = New TextBox
Next
dX = 0
Next
End Sub

Private Sub Button_RetourAccueil_Click(sender As Object, e As EventArgs)
Me.Hide()
Accueil.Show()
End Sub

Public Sub MajTemps()
GestionnaireDePartie.Temps_Partie -= 1
Me.Label_Temps.Text = "Temps restant: " & GestionnaireDePartie.Temps_Partie
GestionnaireDePartie.Fin_Partie(GestionnaireDePartie.Temps_Partie)
End Sub

Private Sub PasDeDelete(ByVal sender As System.Object, ByVal e As
System.Windows.Forms.KeyEventArgs)
If e.KeyCode = Keys.Delete Then
e.SuppressKeyPress = True
End If
End Sub

Private Sub CaractèreAutorisés(ByVal sender As Object, ByVal e As
System.Windows.Forms.KeyPressEventArgs)
Const Arrêt_Validation = 0
Dim i As Integer = Integer.Parse(sender.Name.ToCharArray()(7)),
j As Integer = Integer.Parse(sender.Name.ToCharArray()(8))
Dim toutRempli = True
If e.KeyChar = vbBack Then
sender.Text = ""
ElseIf Not Char.IsDigit(e.KeyChar) Or e.KeyChar = "0" Then
e.Handled = True
Else
sender.Text = e.KeyChar
End If
For di As Integer = 0 To TAILLE_GRILLE - 1
For dj As Integer = 0 To TAILLE_GRILLE - 1
Dim val As String = Grille(di, dj).Text
If Grille(di, dj).Text = "" Then toutRempli = False
If Grille(di, dj).Text = "" OrElse (Grille(di, dj).Enabled AndAlso
VerifColonne(di, dj, val) AndAlso
VerifLigne(di, dj, val) AndAlso VerifSecteur(di, dj, val)) Then
Grille(di, dj).BackColor = Color.LightGray
Grille(di, dj).ForeColor = Color.Black
ElseIf Grille(di, dj).Enabled Then
Grille(di, dj).BackColor = Color.IndianRed
Grille(di, dj).ForeColor = Color.White
```



```
End If
Next
Next
If toutRempli Then Fin_Partie(Arrêt_Validation)
End Sub

Private Sub MajVisible() Handles Me.VisibleChanged
    Me.Text = "Sudoku VB.NET: " & GestionnaireDePartie.JoueurActif.Nom & " à toi
de jouer !"
    Me.CenterToScreen()
End Sub

Private Sub GrilleDeJeu_Closed(sender As Object, e As EventArgs) Handles
Me.Closing
    Const Arrêt_Forcé = -1
    GestionnaireDeDonnées.SauvegardeDonnées()
    Fin_Partie(Arrêt_Forcé)
End Sub

Private Sub Button_Retour_Click(sender As Object, e As EventArgs) Handles
Button_Retour.Click
    Const Arrêt_Forcé = -1
    Timer_Partie.Stop()
    Dim res As Integer = MsgBox("Vous allez perdre votre partie. Êtes-vous sûrs?",
MsgBoxStyle.YesNo, "Quitter")
    If (res = MsgBoxResult.Yes) Then
        Fin_Partie(Arrêt_Forcé)
        Exit Sub
    End If
    Timer_Partie.Start()
End Sub

Private Sub Button_Valider_Click(sender As Object, e As EventArgs)
    Const Arrêt_Validation = 0
    Timer_Partie.Stop()
    Dim res As Integer = MsgBox("Êtes-vous sûrs de vouloir valider votre partie",
MsgBoxStyle.YesNo, "Terminer partie")
    If (res = MsgBoxResult.Yes) Then
        Fin_Partie(Arrêt_Validation)
        Exit Sub
    End If
    Timer_Partie.Start()

End Sub
End Class
```




IV. Les modules et leur code :

I. Module gestionnaire des données :

Imports System.IO

Module GestionnaireDeDonnées

```
Public Class Joueur
    Public Sub New(ByVal Nom As String, ByVal NP As Integer, ByVal MT As Integer, ByVal TT As Integer)
        Me.Nom = Nom
        Me.NbParties = NP
        Me.MeilleurTemps = MT
        Me.TempsTotal = TT
    End Sub

    Public Function GetVals() As String()
        Return {Me.Nom, Me.NbParties, Me.MeilleurTemps, Me.TempsTotal}
    End Function

    Public Sub AddTempsTotal(temps As Integer)
        Me.TempsTotal += temps
    End Sub

    Public Sub MajMeilleurTemps(temps As Integer)
        If Me.MeilleurTemps > temps Or MeilleurTemps = 0 Then Me.MeilleurTemps
= temps
    End Sub

    Public Sub incPartie()
        Me.NbParties += 1
    End Sub

    Public Nom As String
    Public NbParties, MeilleurTemps, TempsTotal As Integer
End Class

Const MAX_JOUEURS = 30
Dim PlayerData As New ArrayList

Public Sub ChargementDonnées()
    If Not File.Exists("./Ressources/PlayerData/Players.txt") Then
        Using create As New StreamWriter("./Ressources/PlayerData/Players.txt",
True)
            create.Write("")
        End Using
    End If
    Using PlayerData_File As New
StreamReader("./Ressources/PlayerData/Players.txt")
        While Not PlayerData_File.EndOfStream
            Dim ligne As String = PlayerData_File.ReadLine
            If Not ligne.Contains(" ") And Not ligne.Length = 0 Then
                Dim ligneSplit As String() = ligne.Split(" ")
                If ligneSplit.Length = 4 AndAlso (IsNumeric(ligneSplit(1))
AndAlso IsNumeric(ligneSplit(2)) AndAlso IsNumeric(ligneSplit(3))) Then
                    PlayerData.Add(New Joueur(ligneSplit(0), ligneSplit(1),
ligneSplit(2), ligneSplit(3)))
                End If
            End If
        End While
    End Using
End Sub
```



```
        End While
    End Using
    If PlayerData.Count = 0 Then AjouterJoueur("Invité")
    If Not PlayerData(0).Nom = "Invité" Then PlayerData.Insert(0, (New
Joueur("Invité", 0, 0, 999)))
    End Sub

    Public Sub SauvegardeDonnées()
        Dim PlayerData_File =
System.IO.File.CreateText("./Ressources/PlayerData/Players.txt")
        Using PlayerData_File
            For Each element As Joueur In PlayerData
                PlayerData_File.WriteLine(Join(element.GetVals, " "))
            Next
        End Using
    End Sub

    Public Sub AjouterJoueur(nom As String)
        PlayerData.Add(New Joueur(nom, 0, 0, 999))
    End Sub

    Public Sub SupprimerJoueur(nom As String)
        For Each p As Joueur In PlayerData
            If nom = p.Nom AndAlso Not nom = PlayerData(0).Nom Then
                PlayerData.Remove(p)
            Exit Sub
        End If
    Next
    End Sub

    Public Function GetDonnées() As ArrayList
        Return PlayerData
    End Function

    Public Function MaxJoueurAtteint() As Boolean
        Return PlayerData.Count >= MAX_JOUEURS
    End Function

    Public Function GetJoueur(nom As String) As Joueur
        For Each e In PlayerData
            If e.Nom = nom Then Return PlayerData(PlayerData.IndexOf(e))
        Next
        Return Nothing
    End Function
End Module
```



2. Module gestionnaire de partie :

Imports System.Timers

Module GestionnaireDePartie

```
Public JoueurActif As Joueur
Public Const TempsPartieHard = 60, CellsHard = 20, Hard = "Difficile"
Public Const TempsPartieAverage = 120, CellsAverage = 35, Average = "Moyen"
Public Const TempsPartieEasy = 500, CellsEasy = 50, Easy = "Facile"
Dim TempsChoisi
Public Temps_Partie As Integer

Public Sub LancerPartie(nomJ As String, difficulté As String)
    TempsChoisi = If(difficulté = Hard, TempsPartieHard, If(difficulté =
Average, TempsPartieAverage, TempsPartieEasy))
    Dim DifficultéCases As Integer = If(difficulté = Hard, CellsHard,
If(difficulté = Average, CellsAverage, CellsEasy))
    Init_Temps(TempsChoisi)
    GrilleDeJeu.Label_Temps.Text = "Temps restant: " & Temps_Partie
    For Each element In GestionnaireDeDonnées.GetDonnées()
        If element.Nom = nomJ Then
            JoueurActif = element
            JoueurActif.incPartie()
        End If
    Next
    GrilleDeJeu.Show()
    GrilleDeJeu.ViderGrille()
    GrilleDeJeu.RemplirGrille(DifficultéCases)
End Sub

Public Sub Init_Temps(temps As Integer)
    Temps_Partie = temps
    GrilleDeJeu.Timer_Partie.Interval = 1000
    GrilleDeJeu.Timer_Partie.Start()
End Sub

Public Sub Fin_Partie(signal As Integer)
    If signal = 0 Or signal = -1 Then
        GrilleDeJeu.Timer_Partie.Stop()
        If Grille_Correct() AndAlso Not signal = -1 Then
            MsgBox("Vous avez gagné la partie !", MsgBoxStyle.OkOnly,
"Information")
            GetJoueur(JoueurActif.Nom).MajMeilleurTemps(TempsChoisi -
Temps_Partie)
        Else
            MsgBox("Vous avez perdu la partie..", MsgBoxStyle.OkOnly,
"Information")
        End If
        GrilleDeJeu.Hide()
        GetJoueur(JoueurActif.Nom).AddTempsTotal(TempsChoisi - Temps_Partie)
        GestionnaireDeDonnées.SauvegardeDonnées()
        Accueil.Show()
    End If
End Sub

Public Function Grille_Correct() As Boolean
    For Each textBox In GrilleDeJeu.Grille
        Dim i As Integer = Integer.Parse(textBox.Name.ToCharArray()(7)),
j As Integer = Integer.Parse(textBox.Name.ToCharArray()(8)),
val As String = textBox.Text
        If Not (VerifColonne(i, j, val) AndAlso VerifLigne(i, j, val) AndAlso
VerifSecteur(i, j, val)) Then
```



```
Return False
End If
Next
Return True
End Function

Public Function VerifColonne(i As Integer, j As Integer, val As String) As
Boolean
    For ligne As Integer = 0 To GrilleDeJeu.Grille.Length / 9 - 1
        If ligne <> i AndAlso GrilleDeJeu.Grille(ligne, j).Text = val Then
            Return False
        End If
    Next
    Return True
End Function

Public Function VerifLigne(i As Integer, j As Integer, val As String) As
Boolean
    For colonne As Integer = 0 To GrilleDeJeu.Grille.Length / 9 - 1
        If colonne <> j AndAlso GrilleDeJeu.Grille(i, colonne).Text = val Then
            Return False
        End If
    Next
    Return True
End Function

Public Function VerifSecteur(i As Integer, j As Integer, val As String) As
Boolean
    Dim iDépart As Integer = If(i < 3, 0, If(i < 6, 3, 6)),
    jDépart As Integer = If(j < 3, 0, If(j < 6, 3, 6))
    For iD As Integer = iDépart To iDépart + 2
        For jD As Integer = jDépart To jDépart + 2
            If iD <> i AndAlso jD <> j AndAlso GrilleDeJeu.Grille(iD, jD).Text
= val Then
                Return False
            End If
        Next
    Next
    Return True
End Function

End Module
```

3. Module Génération automatique de la grille :

Code provenant d'un projet de générateur de grille sudoku open source

Par : The ANZAC, 17 Février 2009

<https://www.codeproject.com/Articles/23206/Sudoku-Algorithm-Generates-a-Valid-Sudoku-in>

V. Représentation de l'enregistrement des informations dans le fichier :

Ceci est un extrait du fichier « Player.txt » où sont enregistrées toutes les données
du jeu :

...

Jean 14 55 585

Luc 22 78 1014

Boseti 4 70 502

Didou 0 999 0

...

Bleu : Nom du joueur

Orange : Nombre de parties jouées

Vert : Meilleur temps

Rouge : Temps total

1	Invité · 75 · 999 · 923
2	Jean · 14 · 55 · 585
3	Luc · 22 · 78 · 1014